Informatica® Cloud Application Integration
April 2025

Amazon Bedrock Connector Guide

Informatica Cloud Application Integration Amazon Bedrock Connector Guide
April 2025

© Copyright Informatica LLC 2024, 2026

Publication Date: 2026-04-22

Table of Contents

Preface 4

Chapter 1: Amazon Bedrock Connector overview 5

Introduction to Amazon Bedrock 5

Chapter 2: Amazon Bedrock connections overview 6

Basic connection properties 6

Amazon Bedrock connection properties 7

Amazon Bedrock connection metadata 7

Publishing Amazon Bedrock connections 8

Using an Amazon Bedrock connection in a process 16

Preface

Read the *Amazon Bedrock Connector Guide* to learn how to set up and use Bedrock connections. This guides assumes that you have an understanding of the Amazon Bedrock API concepts.

CHAPTER 1

Amazon Bedrock Connector overview

Use Amazon Bedrock Connector to integrate, customize, and use advanced foundation models to build powerful, secure, and responsible generative AI applications through a single API.

Amazon Bedrock Connector provides the following features:

- Allows you to evaluate the foundation models and select the best option as per your use case.
- Offers a simple and intuitive API to easily deploy the models into production.
- Ensures privacy of customer data.

Introduction to Amazon Bedrock

Amazon Bedrock (AWS Bedrock) is a fully managed service that offers a choice of high-performing foundation models (FMs) from leading AI companies and Amazon through an API, along with a broad set of capabilities you need to build generative AI applications with security, privacy, and responsible AI.

Use Amazon Bedrock to explore and evaluate leading foundation models for your use case. With Amazon Bedrock's serverless architecture, you can securely integrate and deploy generative AI capabilities into your applications using AWS services.

CHAPTER 2

Amazon Bedrock connections overview

Use an Amazon Bedrock connection to connect programmatically with an AWS service using REST API keys.

After you create an Amazon Bedrock connection, you can validate, test, and save the connection.

You can then publish the Amazon Bedrock connection and click the **Metadata** tab to view the generated process objects for the connection.

Basic connection properties

The following table describes the basic properties that you can configure on the **Properties** tab of the connection creation page:

Property	Description
Name	Required. Unique name for the Amazon Bedrock connection that identifies it in the Process Designer. The name must start with an alphabet and can contain only alphabets, numbers, and hyphens (-).
Location	The location of the project or folder where you want to save the connection. Click **Browse** to select a location. If the **Explore** page is currently active and a project or folder is selected, the default location for the connection is the selected project or folder. Otherwise, the default location is the location of the most recently saved asset.
Description	Optional. Description of the connection.
Type	Required. The type of connection you want to use for the connector or service connector. Select **Amazon Bedrock**.
Runtime Environment	Required. The runtime environment for the connection. You can run the connection on the Cloud Server, a Secure Agent group, or a Secure Agent machine.
Connection Test	Not supported for Amazon Bedrock Connector.
OData-Enabled	Not supported for Amazon Bedrock Connector.

Amazon Bedrock connection properties

To use the Amazon Bedrock, you need an access key, secret key and region.

To create an access key and secret key, go to the AWS Identity and Access Management (IAM) console and create accesses for the corresponding user. Configure the permissions and policies to meet your specific requirements.

The following table defines the Amazon Bedrock connection properties that you must configure in the **Connection Properties** section:

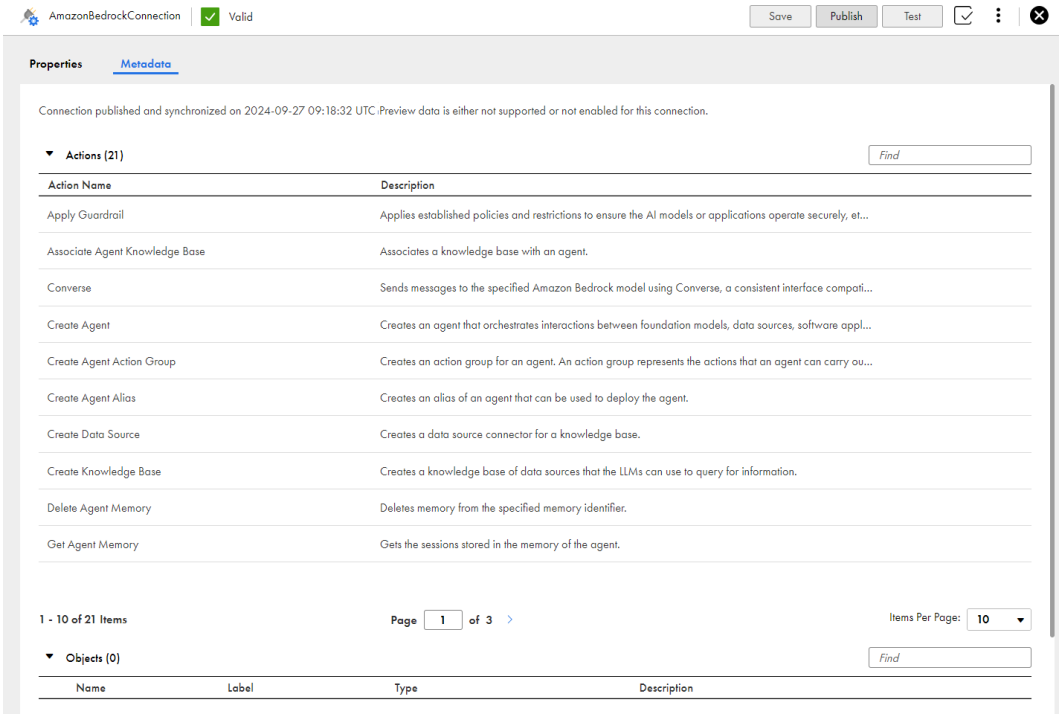
Property	Description
AccessKey	The access key to access Amazon Bedrock.
SecretKey	The secret key to access Amazon Bedrock.
Region	The region that hosts Amazon Bedrock.

Amazon Bedrock connection metadata

After you create an Amazon Bedrock connection, you can validate and save the connection. You can then publish the Amazon Bedrock connection and click the **Metadata** tab to view the generated process objects for the connection.

When you publish an Amazon Bedrock connection in Application Integration, the **Actions** and **Objects** appears on the **Metadata** tab.

The following image shows the **Metadata** tab for a published Amazon Bedrock connection:



Publishing Amazon Bedrock connections

When you publish an Amazon Bedrock connection in Application Integration, the **Actions** and **Objects** appear on the **Metadata** tab.

For the **modelId** property, if you use an inference profile, specify either the inference profile ID or its ARN. For more information, see the AWS Bedrock documentation.

Consider the following information when you use specific actions and objects:

- The **Apply Guardrail** action applies established policies and restrictions to ensure that the AI models and applications comply with the security, ethics, and reliability requirements.
  The following snippet is a sample to build a `requestBody` object:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <source>INPUT</source>
    <content m:isArray="true">
        <text>
            <text> How can I treat a runny nose?</text>
        </text>
    </content>
</root>
```

  If fields in the `requestBody` need to be in different formats, you must add additional namespaces as shown in the following sample code:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <field_array m:isArray="true">
        <property_1> ...</property_1>
    </field_array>
    <field_boolean m:type="xs:boolean">true</field_boolean>
    <field_double m:type="xs:double">20.2</field_double>
    <field_integer m:type="xs:int">10</field_integer>
</root>
```

  For more information about configuring the parameters for the **Apply Guardrail** action, see the Amazon Bedrock documentation.

- The **Associate Agent Knowledge Base** associates a knowledge base with a Secure Agent. If a knowledge base is associated and the indexState is set to **Enabled**, the Secure Agent queries the knowledge base for information to augment its response to the user.
  The following snippet is a sample to build a `requestBody` object:

```
<root>
    <description> Agent description </description>
    <knowledgeBaseId> KnowledgeBaseId </knowledgeBaseId>
<knowledgeBaseState>ENABLED</knowledgeBaseState>
</root>
```

  If fields in the `requestBody` need to be in different formats, you must add additional namespaces as shown in the following sample code:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <field_array m:isArray="true">
        <property_1> ...</property_1>
    </field_array>
    <field_boolean m:type="xs:boolean">true</field_boolean>
    <field_double m:type="xs:double">20.2</field_double>
    <field_integer m:type="xs:int">10</field_integer>
</root>
```

  For more information about configuring the parameters for the **Associate Agent Knowledge Base** action, see the Amazon Bedrock documentation.

- The **Converse** action sends messages to the specified Amazon Bedrock model using a consistent interface that is compatible with all message-supporting models. This enables you to write code once and reuse it with different models.

The following snippet is a sample to build a `requestBody` object:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <anthropic_version>bedrock-2023-05-31</anthropic_version>
    <max_tokens m:type="xs:int">50</max_tokens>
    <messages m:isArray="true">
        <role>user</role>
        <content m:isArray="true">
            <type>text</type>
            <text>Tell me a joke</text>
        </content>
    </messages>
</root>
```

If fields in the `requestBody` need to be in different formats, you must add additional namespaces as shown in the following sample code:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <field_array m:isArray="true">
        <property_1> ...</property_1>
    </field_array>
    <field_boolean m:type="xs:boolean">true</field_boolean>
    <field_double m:type="xs:double">20.2</field_double>
    <field_integer m:type="xs:int">10</field_integer>
</root>
```

For more information about configuring the parameters for the **Converse** action, see the Amazon Bedrock documentation.

- The **Create Agent** creates an agent that orchestrates interactions between foundation models, data sources, software applications, user conversations, and APIs for customer assistance.
  To create and use Agent, use the actions in the following order:

  1. Create Agent
  2. Associate Agent Knowledge Base
  3. Create Agent Action Group
  4. Update Agent
  5. Prepare Agent
  6. Create Agent Alias

  The following snippet is a sample to build a `requestBody` object:

```
<root>
    <agentName>AgentName</agentName>
    <instruction> Agent instruction </instruction>
    <description> Agent description </description>
    <foundationModel>anthropic.claude-v2</foundationModel>
    <agentResourceRoleArn>arn:aws:iam::800000000000:role/ROLE  </agentResourceRoleArn>
</root>
```

  If fields in the `requestBody` need to be in different formats, you must add additional namespaces as shown in the following sample code:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <field_array m:isArray="true">
        <property_1> ...</property_1>
    </field_array>
    <field_boolean m:type="xs:boolean">true</field_boolean>
    <field_double m:type="xs:double">20.2</field_double>
    <field_integer m:type="xs:int">10</field_integer>
</root>
```

  For more information about configuring the parameters for the **Create Agent** action, see the Amazon Bedrock documentation.

- The **Create Agent Action Group** creates an action group for a Secure Agent. An action group represents the actions that a Secure Agent can carry out for the customer by defining the APIs that a Secure Agent can call and the logic for calling them.
  The following snippet is a sample to build a `requestBody` object:

```
<root>
    <actionGroupExecutor>
        <lambda>arn:aws:lambda:us-east-1:800000000000:function: functionName</lambda>
    </actionGroupExecutor>
    <actionGroupName> ActionGroupName </actionGroupName>
    <apiSchema>
        <s3>
            <s3BucketName>Bucket</s3BucketName>
            <s3ObjectKey>openAPI.json</s3ObjectKey>
        </s3>
    </apiSchema>
</root>
```

  If fields in the `requestBody` need to be in different formats, you must add additional namespaces as shown in the following sample code:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <field_array m:isArray="true">
        <property_1> ...</property_1>
    </field_array>
    <field_boolean m:type="xs:boolean">true</field_boolean>
    <field_double m:type="xs:double">20.2</field_double>
    <field_integer m:type="xs:int">10</field_integer>
</root>
```

  For more information about configuring the parameters for the **Create Agent Action Group** action, see the Amazon Bedrock documentation.

- The **Create Agent Alias** creates an alias for a Secure Agent that can be used to deploy the agent.
  The following snippet is a sample to build a `requestBody` object:

```
<root>
    <agentAliasName>AliasName</agentAliasName>
</root>
```

  If fields in the `requestBody` need to be in different formats, you must add additional namespaces as shown in the following sample code:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <field_array m:isArray="true">
        <property_1> ...</property_1>
    </field_array>
    <field_boolean m:type="xs:boolean">true</field_boolean>
    <field_double m:type="xs:double">20.2</field_double>
    <field_integer m:type="xs:int">10</field_integer>
</root>
```

  For more information about configuring the parameters for the **Create Agent Alias** action, see the Amazon Bedrock documentation.

- The **Create Data Source** action creates a data source connector for a knowledge base.
  The following snippet is a sample to build a `requestBody` object:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <dataDeletionPolicy>DELETE</dataDeletionPolicy>
    <dataSourceConfiguration>
        <s3Configuration>
            <bucketArn>arn:aws:s3:::BUCKET</bucketArn>
            <inclusionPrefixes m:isArray="true">
Lorem_ipsum.pdf

</inclusionPrefixes>
        </s3Configuration>
        <type>S3</type>
    </dataSourceConfiguration>
```

```
    <name>Source</name>
</root>
```

If fields in the `requestBody` need to be in different formats, you must add additional namespaces as shown in the following sample code:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <field_array m:isArray="true">
        <property_1> ...</property_1>
    </field_array>
    <field_boolean m:type="xs:boolean">true</field_boolean>
    <field_double m:type="xs:double">20.2</field_double>
    <field_integer m:type="xs:int">10</field_integer>
</root>
```

For more information about configuring the parameters for the **Create Data Source** action, see the Amazon Bedrock documentation.

- The **Create Knowledge Base** action creates a knowledge base of data sources that the LLMs can use to query for information.
  To create and use the Knowledge Base, use the actions in the following order::
  1. Create Knowledge Base
  2. Create Data Source
  3. Start Ingestion Job
  4. Get List of Ingestion Jobs
  5. Retrieve and Generate Knowledge Base

  The following snippet is a sample to build a `requestBody` object:

```
<root>
    <knowledgeBaseConfiguration>
        <type>VECTOR</type>
        <vectorKnowledgeBaseConfiguration>
            <embeddingModelArn>arn:aws:bedrock:us-east-1::foundation-model/
amazon.titan-embed-text-v1</embeddingModelArn>
        </vectorKnowledgeBaseConfiguration>
    </knowledgeBaseConfiguration>
    <name> KnowledgeBaseName </name>
    <roleArn>arn:aws:iam::800000000000:role/service-role/
AmazonBedrockExecutionRoleForKnowledgeBase_ROLE</roleArn>
    <storageConfiguration>
        <opensearchServerlessConfiguration>
            <collectionArn>arn:aws:aoss:us-east-1: 800000000000:collection/
COLLECTION</collectionArn>
            <fieldMapping>
                <metadataField>metadata</metadataField>
                <textField>text</textField>
                <vectorField>name</vectorField>
            </fieldMapping>
            <vectorIndexName>index</vectorIndexName>
        </opensearchServerlessConfiguration>
        <type>OPENSEARCH_SERVERLESS</type>
    </storageConfiguration>
</root>
```

If fields in the `requestBody` need to be in different formats, you must add additional namespaces as shown in the following sample code:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <field_array m:isArray="true">
        <property_1> ...</property_1>
    </field_array>
    <field_boolean m:type="xs:boolean">true</field_boolean>
    <field_double m:type="xs:double">20.2</field_double>
    <field_integer m:type="xs:int">10</field_integer>
</root>
```

For more information about configuring the parameters for the **Create Knowledge Base** action, see the Amazon Bedrock documentation.

- The **Delete Agent Memory** action deletes memory from the specified memory identifier. You can delete all stored sessions when memory is enabled for your Secure Agent.
  After a session ends, the Secure Agent summarizes the conversation and stores the conclusion in its memory. This process might take some time.

  For more information about configuring the parameters for the **Delete Agent Memory** action, see the Amazon Bedrock documentation.
- The **Get Agent Memory** action returns the sessions stored in the Secure Agent's memory. You can view these stored sessions when memory is enabled for your agent.
  After a session ends, the Secure Agent summarizes the conversation and stores the conclusion in its memory. This process might take some time.

  For more information about configuring the parameters for the **Get Agent Memory** action, see the Amazon Bedrock documentation.
- Use the **Get Foundation Models List** action to get a list of the available models.
  For more information about using the request parameter URI, see the Amazon Bedrock documentation.
- The **Get List Ingestion Jobs** gets a list of ingestion jobs for a data source and provides information about each job.
  If fields in the `requestBody` need to be in different formats, you must add additional namespaces as shown in the following sample code:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <field_array m:isArray="true">
        <property_1> ...</property_1>
    </field_array>
    <field_boolean m:type="xs:boolean">true</field_boolean>
    <field_double m:type="xs:double">20.2</field_double>
    <field_integer m:type="xs:int">10</field_integer>
</root>
```

  For more information about configuring the parameters for the **Get List Ingestion Jobs** action, see the Amazon Bedrock documentation.
- The **Invoke Agent** action in Amazon Bedrock Agents allow you to build and configure autonomous agents within your application. These agents assist your end users in completing actions based on organizational data and user input. Assign a new value to `sessionId` each time you start a new dialog with the agent. Set a unique identifier in the `memoryId` field within the request body when memory is enabled for the Secure Agent.
  Secure Agent responses are provided in the base64 format. To decode the response, set `decodeOutput` to `true`. There is no option to enable trace for tracking the agent's reasoning process.

  The following snippet is a sample to build a `requestBody` object:

```
<root>
    <inputText>Send email to employee Alex </inputText>
    <memoryId>Memory_01</memoryId>
</root>
```

  If fields in the `requestBody` need to be in different formats, you must add additional namespaces as shown in the following sample code:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <field_array m:isArray="true">
        <property_1> ...</property_1>
    </field_array>
    <field_boolean m:type="xs:boolean">true</field_boolean>
    <field_double m:type="xs:double">20.2</field_double>
    <field_integer m:type="xs:int">10</field_integer>
</root>
```

For more information about configuring the parameters for the **Invoke Agent** action, see the Amazon Bedrock documentation.

- The **Invoke Flow** action runs a flow alias to process the inputs that you specify and returns the output from each node.
  The **finalResponse** contains the value `document` in the **FlowOutputContent** object.

  The following snippet is a sample to build a `requestBody` object:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <inputs  m:isArray="true">
        <nodeName>FlowInputNode</nodeName>
        <nodeOutputName>document</nodeOutputName>
        <content>
            <document>Some text</document>
        </content>
    </inputs>
</root>
```

  If fields in the `requestBody` need to be in different formats, you must add additional namespaces as shown in the following sample code:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <field_array m:isArray="true">
        <property_1> ...</property_1>
    </field_array>
    <field_boolean m:type="xs:boolean">true</field_boolean>
    <field_double m:type="xs:double">20.2</field_double>
    <field_integer m:type="xs:int">10</field_integer>
</root>
```

  For more information about configuring the parameters for the **Invoke Flow** action, see the Amazon Bedrock documentation.

- The **Invoke Model** action invokes the specified Amazon Bedrock model to run inference using the prompt and parameters provided in the request body. It uses model inference to generate text, images, and embeddings. This action does not support Guardrail to call models. The prompt and inference parameters in the `requestBody` vary for each model.
  The following snippet is a sample `requestBody` object:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <prompt>Human: Tell me a joke.Assistant:</prompt>
    <max_tokens_to_sample m:type="xs:int">100</max_tokens_to_sample>
</root>
```

  For more information about the inference parameters for foundation models, see the Amazon Bedrock documentation.

  If fields in the request body need to be in different formats, you must add additional namespaces as shown in the following sample code:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <field_array m:isArray="true">
        <property_1> ...</property_1>
    </field_array>
    <field_boolean m:type="xs:boolean">true</field_boolean>
    <field_double m:type="xs:double">20.2</field_double>
    <field_integer m:type="xs:int">10</field_integer>
</root>
```

  For more information about configuring the parameters for the **Invoke Model** action, see the Amazon Bedrock documentation.

- The **Invoke Model with Guardrail** action invokes the specified Amazon Bedrock model to run inference using the prompt and parameters provided in the request body. It uses model inference to generate text,

images, and embeddings, and applies a guardrail. The prompt and inference parameters in the `requestBody` vary for each model.
The following snippet is a sample `requestBody` object:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <prompt>Human: Tell me a joke.Assistant:</prompt>
    <max_tokens_to_sample m:type="xs:int">100</max_tokens_to_sample>
</root>
```

For more information about the inference parameters for foundation models, see the Amazon Bedrock documentation.

If fields in the `requestBody` need to be in different formats, you must add additional namespaces as shown in the following sample code:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <field_array m:isArray="true">
        <property_1> ...</property_1>
    </field_array>
    <field_boolean m:type="xs:boolean">true</field_boolean>
    <field_double m:type="xs:double">20.2</field_double>
    <field_integer m:type="xs:int">10</field_integer>
</root>
```

For more information about configuring the parameters for the **Invoke Model with Guardrail** action, see the Amazon Bedrock documentation.

- The **Prepare Agent** creates a draft version of the Secure Agent for internal testing.
  For more information about configuring the parameters for the **Prepare Agent** action, see the Amazon Bedrock documentation.
- The **Retrieve and Generate Knowledge Base** action generates responses based on the queried knowledge base results. The response includes only sources relevant to the query.
  The following snippet is a sample to build a `requestBody` object:

```
<root>
    <input>
        <text>Give me the email of the employee Alex</text>
    </input>
    <retrieveAndGenerateConfiguration>
        <knowledgeBaseConfiguration>
        <knowledgeBaseId> KnowledgeBaseId </knowledgeBaseId>
            <modelArn>anthropic.claude-v2:1</modelArn>
        </knowledgeBaseConfiguration>
        <type>KNOWLEDGE_BASE</type>
    </retrieveAndGenerateConfiguration>
</root>
```

  If fields in the `requestBody` need to be in different formats, you must add additional namespaces as shown in the following sample code:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <field_array m:isArray="true">
        <property_1> ...</property_1>
    </field_array>
    <field_boolean m:type="xs:boolean">true</field_boolean>
    <field_double m:type="xs:double">20.2</field_double>
    <field_integer m:type="xs:int">10</field_integer>
</root>
```

  For more information about configuring the parameters for the **Retrieve and Generate Knowledge Base** action, see the Amazon Bedrock documentation.
- The **Retrieve Knowledge Base** action queries a knowledge base to fetch information.

The following snippet is a sample to build a `requestBody` object:

```
<root>
    <retrievalQuery>
        <text> Give me the email of the employee Alex</text>
    </retrievalQuery>
</root>
```

If fields in the `requestBody` need to be in different formats, you must add additional namespaces as shown in the following sample code:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <field_array m:isArray="true">
        <property_1> ...</property_1>
    </field_array>
    <field_boolean m:type="xs:boolean">true</field_boolean>
    <field_double m:type="xs:double">20.2</field_double>
    <field_integer m:type="xs:int">10</field_integer>
</root>
```

For more information about configuring the parameters for the **Retrieve Knowledge Base** action, see the Amazon Bedrock documentation.

- The **Start Ingestion Job** starts an ingestion job to add a data source to a knowledge base.
  If fields in the `requestBody` need to be in different formats, you must add additional namespaces as shown in the following sample code:

```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <field_array m:isArray="true">
        <property_1> ...</property_1>
    </field_array>
    <field_boolean m:type="xs:boolean">true</field_boolean>
    <field_double m:type="xs:double">20.2</field_double>
    <field_integer m:type="xs:int">10</field_integer>
</root>
```

  For more information about configuring the parameters for the **Start Ingestion Job** action, see the Amazon Bedrock documentation.

- The **Update Agent** updates the configuration of a Secure Agent.
  The following snippet is a sample to build a `requestBody` object:

```
<root>
    <agentName>Review</agentName>
    <agentResourceRoleArn>arn:aws:iam::800000000000:role/ROLE</agentResourceRoleArn>
    <foundationModel>anthropic.claude-v2</foundationModel>
    <instruction> Agent instruction </instruction>
    <description> Agent description </description>
    <promptOverrideConfiguration>
        <promptConfigurations>
            <promptCreationMode>DEFAULT</promptCreationMode>
            <promptType>KNOWLEDGE_BASE_RESPONSE_GENERATION</promptType>
        </promptConfigurations>
        <promptConfigurations>
            <promptCreationMode>DEFAULT</promptCreationMode>
            <promptType>POST_PROCESSING</promptType>
        </promptConfigurations>
        <promptConfigurations>
            <promptCreationMode>DEFAULT</promptCreationMode>
            <promptType>ORCHESTRATION</promptType>
        </promptConfigurations>
        <promptConfigurations>
            <promptCreationMode>DEFAULT</promptCreationMode>
            <promptType>PRE_PROCESSING</promptType>
        </promptConfigurations>
    </promptOverrideConfiguration>
</root>
```

If fields in the `requestBody` need to be in different formats, you must add additional namespaces as shown in the following sample code:

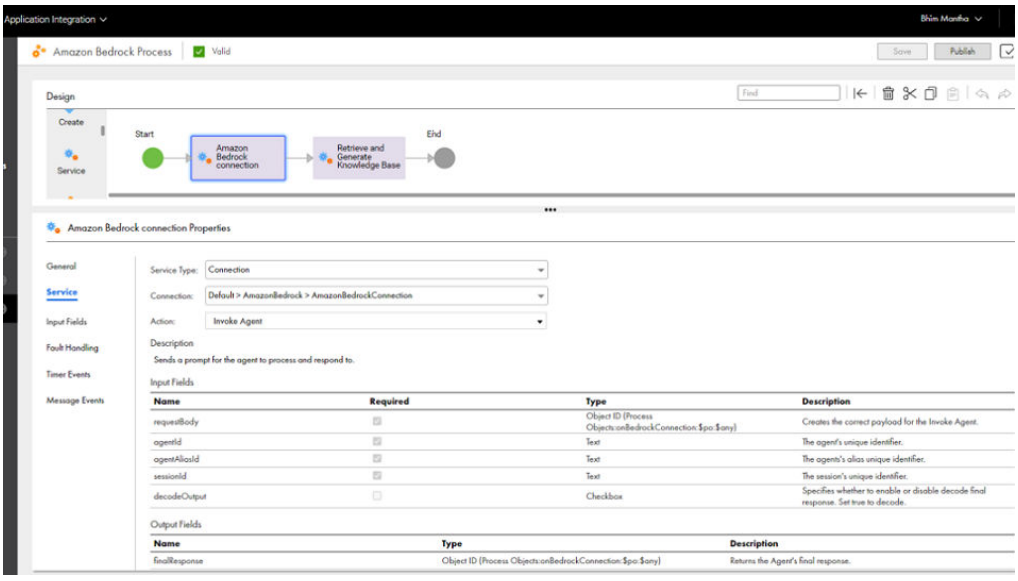
```
<root xmlns:m="urn:informatica:ae:xquery:json2xml:meta-data">
    <field_array m:isArray="true">
        <property_1> ...</property_1>
    </field_array>
    <field_boolean m:type="xs:boolean">true</field_boolean>
    <field_double m:type="xs:double">20.2</field_double>
    <field_integer m:type="xs:int">10</field_integer>
</root>
```

For more information about configuring the parameters for the **Update Agent** action, see the Amazon Bedrock documentation.

Using an Amazon Bedrock connection in a process

After you create an Amazon Bedrock connection, you can use it in a Service step in a process.

1. Add a Service step to the process.
2. Click the **Service** tab.
3. From the **Service Type** list, select **Connection**.
4. From the **Connection** list, browse and select the Amazon Bedrock connection that you created.
5. From the **Action** list, select the action that you want to perform. For example, select the **Invoke Agent** action to send a prompt for the agent to process and respond to shown in the following image:



   In this case, Application Integration populates the **finalResponse** object under the output fields.
6. Configure further steps as needed.
7. Validate, save, and publish the process.